WEEK 5 – PAYMENT MANAGEMENT REPORT

Flower Shop Management System

1. Objective

To insert payment details and perform SQL queries to analyze payment status, payment modes, transaction amounts, revenue, and customer order-payment details.

CREATE PAYMENT TABLE

```
SQL> CREATE TABLE PAYMENT(PAYMENT_ID VARCHAR(7),ORDER_ID VARCHAR(10),PAYMENT
_DATE DATE,PAYMENT_MODE VARCHAR(20),PAYMENT NUMBER(10,2));

Table created.
```

2. Primary Key Constraint

```
SQL> ALTER TABLE PAYMENT ADD CONSTRAINT pk_PAYMENT PRIMARY KEY (PAYMENT_ID);

Table altered.
```

Ensures that each PAYMENT_ID is unique and cannot be NULL.

3. Foreign Key Constraint

```
SQL> ALTER TABLE PAYMENT ADD CONSTRAINT fk_PAYMENT_ORDER FOREIGN KEY (ORDER_
ID) REFERENCES ORDERS(ORDER_ID);

Table altered.
```

Purpose: Links the PAYMENT table with the ORDERS table. The ORDER_ID entered in PAYMENT must exist in ORDERS.

3. Payment Mode Check Constraint

```
SQL> ALTER TABLE PAYMENT ADD CONSTRAINT chk_PAYMENT_MODE CHECK (PAYMENT_MODE IN ('UPI','CARD','NET BANKING','CASH'));
Table altered.
SQL> ALTER TABLE PAYMENT ADD CONSTRAINT chk_PAYMENT_STATUS CHECK (PAYMENT_STATUS IN ('COMPLETED','FAILED','PENDING'));
Table altered.
```

Purpose: Allows only UPI, CARD, NET BANKING, or CASH as payment modes.

```
SQL> ALTER TABLE PAYMENT ADD CONSTRAINT chk_TRANSACTION_AMOUNT CHECK (TRANSACTION_AMOUNT >0);

Table altered.
```

2. Payment Record Insertion

```
SQL> INSERT INTO PAYMENT (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_MODE,
PAYMENT_STATUS, TRANSACTION_AMOUNT)  VALUES ('PAY101', '0001', TO_DATE('2026
-09-03','YYYY-MM-DD'), 'UPI', 'COMPLETED', 3099);

1 row created.

SQL> INSERT INTO PAYMENT (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_MODE,
PAYMENT_STATUS, TRANSACTION_AMOUNT)  VALUES ('PAY102', '0002', TO_DATE('2026
-09-04','YYYY-MM-DD'), 'CARD', 'FAILED', 1500);

1 row created.

SQL> INSERT INTO PAYMENT (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_MODE,
PAYMENT_STATUS, TRANSACTION_AMOUNT)  VALUES ('PAY103', '0003', TO_DATE('2026
-09-09','YYYY-MM-DD'), 'NET BANKING', 'PENDING', 3500);

1 row created.

SQL> INSERT INTO PAYMENT (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_MODE,
PAYMENT_STATUS, TRANSACTION_AMOUNT)  VALUES ('PAY104', '0004', TO_DATE('2026
-12-25','YYYY-MM-DD'), 'CARD', 'FAILED', 4500);

1 row created.

SQL> INSERT INTO PAYMENT (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_MODE,
PAYMENT_STATUS, TRANSACTION_AMOUNT)  VALUES ('PAY105', '0005', TO_DATE('2008
-08-14','YYYY-MM-DD'), 'CASH', 'COMPLETED', 4500);

1 row created.
```

The following query is used to insert a completed UPI payment into the PAYMENT table:

3. Payment Table Queries

Display all payment records

```
SQL> SELECT * FROM PAYMENT;

PAYMENT_ID ORDER_ID   PAYMENT_DATE PAYMENT_MODE      PAYMENT PAYMENT_STATUS
 TRANSACTION_AMOUNT
---------- ---------- ------------ -------------- --------- --------------
 ------------------
PAY101     0001       03-SEP-26    UPI                      COMPLETED     3
099.00
PAY102     0002       04-SEP-26    CARD                     FAILED        1
500.00
PAY103     0003       09-SEP-26    NET BANKING              PENDING       3
500.00
PAY104     0004       25-DEC-26    CARD                     FAILED        4
500.00
PAY105     0005       14-AUG-08    CASH                     COMPLETED     4
500.00
```

Display completed payments

```
SQL> SELECT * FROM PAYMENT WHERE PAYMENT_STATUS = 'COMPLETED';

PAYMENT_ID ORDER_ID   PAYMENT_DATE PAYMENT_MODE      PAYMENT PAYMENT_STATUS
---------- ---------- ------------ --------------- --------- ---------------
TRANSACTION_AMOUNT
------------------
PAY101     0001       03-SEP-26    UPI                       COMPLETED
           3099.00

PAY105     0005       14-AUG-08    CASH                      COMPLETED
           4500.00
```

Display failed payments

```
SQL> SELECT * FROM PAYMENT WHERE PAYMENT_STATUS = 'FAILED';

PAYMENT_ID ORDER_ID   PAYMENT_DATE PAYMENT_MODE      PAYMENT PAYMENT_STATUS
 TRANSACTION_AMOUNT
---------- ---------- ------------ --------------- --------- ---------------
 ------------------
PAY102     0002       04-SEP-26    CARD                      FAILED        1
500.00
PAY104     0004       25-DEC-26    CARD                      FAILED        4
500.00
```

Count payments by status

```
SQL> SELECT PAYMENT_STATUS, COUNT(PAYMENT_STATUS) FROM PAYMENT GROUP BY PAYM
ENT_STATUS;

PAYMENT_STATUS  COUNT(PAYMENT_STATUS)
--------------- --------------------
PENDING                            1
COMPLETED                          2
FAILED                             2
```

Count UPI transactions

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTIONS FROM PAYMENT WHERE PAYMENT_MODE = '
UPI';

UPI_TRANSACTIONS
----------------
               1
```

Count card payments

```
SQL> SELECT COUNT (*) AS CARD_PAYMENTS FROM PAYMENT WHERE PAYMENT_MODE = 'CARD';

CARD_PAYMENTS
-------------
            2
```

Count transactions by payment mode

```
SQL> SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE ORDER BY TRANSACTION_COUNT DESC;

PAYMENT_MODE    TRANSACTION_COUNT
--------------- -----------------
CARD                            2
CASH                            1
UPI                             1
NET BANKING                     1
```

Calculate total completed revenue

```
SQL> SELECT SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE FROM PAYMENT WHERE PAYMENT_STATUS = 'COMPLETED';

TOTAL_REVENUE
-------------
         7599
```

Revenue by completed payment mode

```
SQL> SELECT  PAYMENT_MODE,SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE FROM PAYMENT WHERE PAYMENT_STATUS = 'COMPLETED' GROUP BY PAYMENT_MODE ORDER BY TOTAL_REVE
NUE DESC;

PAYMENT_MODE    TOTAL_REVENUE
--------------- -------------
CASH                     4500
UPI                      3099
```

Average completed transaction amount

```
SQL> SELECT ROUND(AVG(TRANSACTION_AMOUNT), 2) AS AVERAGE_TRANSACTION_AMOUNT FROM PAYMENT WHERE PAYMENT_STATUS = 'COMPLETED';

AVERAGE_TRANSACTION_AMOUNT
--------------------------
                    3799.5
```

Payment mode summary

```
SQL> SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT, SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE, ROUND(AVG(TRANSACTION_AMOUNT), 2) AS AVG_TRANSACTION FROM
 PAYMENT WHERE PAYMENT_STATUS = 'COMPLETED' GROUP BY PAYMENT_MODE ORDER BY TOTAL_REVENUE DESC;

PAYMENT_MODE    TRANSACTION_COUNT TOTAL_REVENUE AVG_TRANSACTION
--------------- ----------------- ------------- --------------
CASH                            1          4500           4500
UPI                             1          3099           3099
```

4. Customer, Order and Payment Report

The following JOIN query combines CUSTOMER, ORDERS, and PAYMENT tables to display customer name, order details, payment date, payment mode, amount, and payment status:

```
SQL> SELECT C.CUSTOMER_NAME,O.ORDER_ID,O.ORDER_DATE,P.PAYMENT_DATE,P.PAYMENT_MODE,P.TRANSACTION_AMOUNT AS AMOUNT,P.PAYMENT_STATUS
  2  FROM CUSTOMER C
  3  JOIN ORDERS O ON C.CUSTOMER_ID=O.CUSTOMER_ID
  4  JOIN PAYMENT P ON O.ORDER_ID=P.ORDER_ID
  5  ORDER BY P.PAYMENT_DATE;

CUSTOMER_NAM ORDER_ID ORDER_DATE   PAYMENT_DATE PAYMENT_MODE    AMOUNT PAYMENT_STAT
------------ -------- ------------ ------------ --------------- ------ ------------
Gobi         0005     31-JUL-08    14-AUG-08    CASH              4500 COMPLETED
Ananya       0001     01-SEP-26    03-SEP-26    UPI               3099 COMPLETED
Priya        0002     03-SEP-26    04-SEP-26    CARD              1500 FAILED
Kaviya       0003     05-SEP-26    09-SEP-26    NET BANKING       3500 PENDING
Karu         0004     23-DEC-26    25-DEC-26    CARD              4500 FAILED
```